iR-compensation-calculation

A **Streamlit GUI** for automated ohmic-drop (*iR*) compensation of linear-sweep voltammetry (LSV) data, using the uncompensated resistance **Ru** estimated from electrochemical impedance spectroscopy (EIS).

■ **Live app:** https://lsv-ir-compensation-calculation.streamlit.app/

> Estimate `Ru` from an EIS Nyquist arc, then apply a user-selected iR-compensation factor (5 – 85 %) to your LSV curve and export the corrected data.

Workflow

```
Excel / CSV ■■  EIS (Z', Z'')  ■■  circle fit  ■■  Ru  ■
                                                  ■■■  E_corr = E – (f%)·I·Ru  ■■  CSV
              LSV (Potential, Current)  ■■■■■■■■■■■■■■■■■
```

1. **Load data** – an Excel workbook where **Sheet 1 = EIS** (`Z'`, `Z''`) and **Sheet 2 = LSV** (`Potential`, `Current`), or two separate CSV files. Each sheet may hold **several datasets side-by-side as repeated column pairs** (`Z'_1, Z''_1, Z'_2, Z''_2, …` and `E_1, I_1, E_2, I_2, …`). The app parses every pair and **links EIS sample *i* to LSV sample *i***; pick the active sample from the sidebar.
2. **EIS / Ru Analysis tab** – the kinetic semicircle of the Nyquist plot is fitted with an algebraic circle fit. Its **high-frequency real-axis intercept is Ru**; the low-frequency intercept gives `Ru + Rct`. The low-frequency diffusion tail is auto-excluded and the fit range is fully adjustable. You can also enter Ru manually. With multiple samples loaded, a **batch table** lists the fitted `Ru` for every sample.
3. **LSV iR Correction tab** – choose the current unit and one *or several* compensation factors (5 – 85 %). The app computes

```
text E_corrected = E_measured – (factor / 100) · I · Ru
```

shows a **with-vs-without comparison** (overlay or side-by-side) plus the applied iR drop, and lets you **download a CSV** that embeds `Ru`, the compensation %, and one column block per factor. 4. **Over-compensation guard** – each factor is checked for *fold-back* (the corrected potential reversing instead of advancing). Over-corrected factors are flagged with a ■ status, and the app reports the **highest safe factor** for that sample plus an interpretation of what a good compensation looks like.

Why Ru comes from a circle fit (not an equivalent-circuit fit)

The reference dataset stores only `Z'` and `Z''` with **no frequency column**, so a frequency-dependent equivalent-circuit (e.g. Randles) fit is not possible. The geometric circle fit recovers Ru directly from the Nyquist arc and needs no frequency data.

Installation

Requires Python 3.10+.

```
pip install -r requirements.txt
```

Dependencies: `numpy`, `scipy`, `pandas`, `openpyxl`, `streamlit`, `plotly`.

Running the app

```
streamlit run app.py
```

Then open the URL shown in the terminal (default `http://localhost:8501`). Select **Sample workbook** in the sidebar to try it immediately with `sample-data/Book1.xlsx`.

Deploy publicly on Streamlit Community Cloud

This app is ready to deploy for free on Streamlit Community Cloud:

1. Push this repository to GitHub (it already contains `app.py` and `requirements.txt` at the root, which is all the platform needs).
2. Go to **share.streamlit.io** → **Create app**, pick this repo/branch, and set the **main file path** to `app.py`.
3. Click **Deploy**. The platform installs `requirements.txt` and serves the app at `https://<your-app-name>.streamlit.app`.

The sample workbook ships in `sample-data/`, so the deployed app works out of the box; users can also upload their own Excel/CSV files.

Keeping the source code private

Streamlit Community Cloud runs the app **directly from the GitHub repo it is linked to**, so the code must live in that repo — you cannot `.gitignore` `app.py` and still deploy it (the platform would not have the file). To publish a working app while keeping the source hidden from the public, **make the GitHub repository private**: Community Cloud can deploy from private repos (authorize repo access when creating the app). The deployed app stays publicly reachable at its `*.streamlit.app` URL, while the code is not visible to anyone browsing GitHub. (You can additionally restrict app *viewers* in the app's **Settings** → **Sharing** if you also want to limit who can use it.)

Project layout

Path	Purpose
`app.py`	Streamlit GUI (EIS/Ru analysis + LSV iR-correction tabs).
`ir_compensation/data_io.py`	Load EIS / LSV from Excel or CSV (fuzzy column matching).

`ir_compensation/eis.py`	Circle fit of the Nyquist arc → `Ru`, `Rct`.
`ir_compensation/correction.py`	Apply the iR correction (5–85 % factor).
`sample-data/Book1.xlsx`	Example data: Sheet 1 EIS, Sheet 2 LSV.

Using the core API without the GUI

```
from ir_compensation import data_io, eis, correction

eis_d = data_io.load_eis("sample-data/Book1.xlsx", sheet=0)
lsv_d = data_io.load_lsv("sample-data/Book1.xlsx", sheet=1)

ru = eis.fit_ru_circle(eis_d.z_real, eis_d.z_imag).ru     # ≈ 27.5 Ω
result = correction.apply_ir_correction(
   lsv_d.potential, lsv_d.current, ru,
   factor_percent=85, current_unit="mA",
)
result.potential_corrected   # iR-corrected potentials (V)
```

Notes

- **Current units** (`A`, `mA`, `µA`, `nA`) are selectable so that `I·Ru` resolves to volts; pick the unit your file actually uses.
- Compensation is intentionally capped at **85 %** — full (100 %) positive feedback can over-correct and induce oscillation; partial compensation is safer and is the project default.

Security

Because the app accepts arbitrary uploaded files on a public URL, several safeguards are built in:

- **Upload limits.** `.streamlit/config.toml` caps upload size (15 MB) and message size, keeps XSRF protection on, disables usage telemetry, and hides Python tracebacks from end users (`showErrorDetails = false`).
- **Malicious-file defenses** (`ir_compensation/data_io.py`): Excel archives are checked for **decompression-bomb** expansion before parsing; tables are rejected above row/column caps (memory-exhaustion / DoS guard); the number of parsed column-pairs is capped; and header-derived labels are stripped of control characters / newlines before display or CSV output (prevents CSV/markdown injection).
- **No persisted data.** Uploads are processed in memory only; nothing is written to disk. The sample is **not preloaded**, and a **Clear / reset** button wipes session data.
- **Optional password gate.** Set an `app_password` secret (see `.streamlit/secrets.toml.example`) to require a password; comparison is constant-time. Leave it unset for a fully public app. `secrets.toml` is git-ignored, so credentials are never committed.

To restrict *who* can open the app entirely, you can also limit viewers under the app's **Settings** → **Sharing** on Streamlit Community Cloud.

Citation

If you use this app (including a public Streamlit deployment) in your work, please cite it. Machine-readable metadata is in `CITATION.cff`; a plain-text form:

> Kumar, R. (2026). *Automated iR Compensation: EIS fitting and LSV correction* (v1.0.0) [Computer software]. North Carolina Central University. https://github.com/rajeev4187/LSV-iR-compensation-calculation

```
@software{kumar_ir_compensation_2026,
 author  = {Kumar, Rajeev},
 title   = {Automated iR Compensation: EIS fitting and LSV correction},
 version = {1.0.0},
 year    = {2026},
 url      = {https://github.com/rajeev4187/LSV-iR-compensation-calculation}
}
```

License